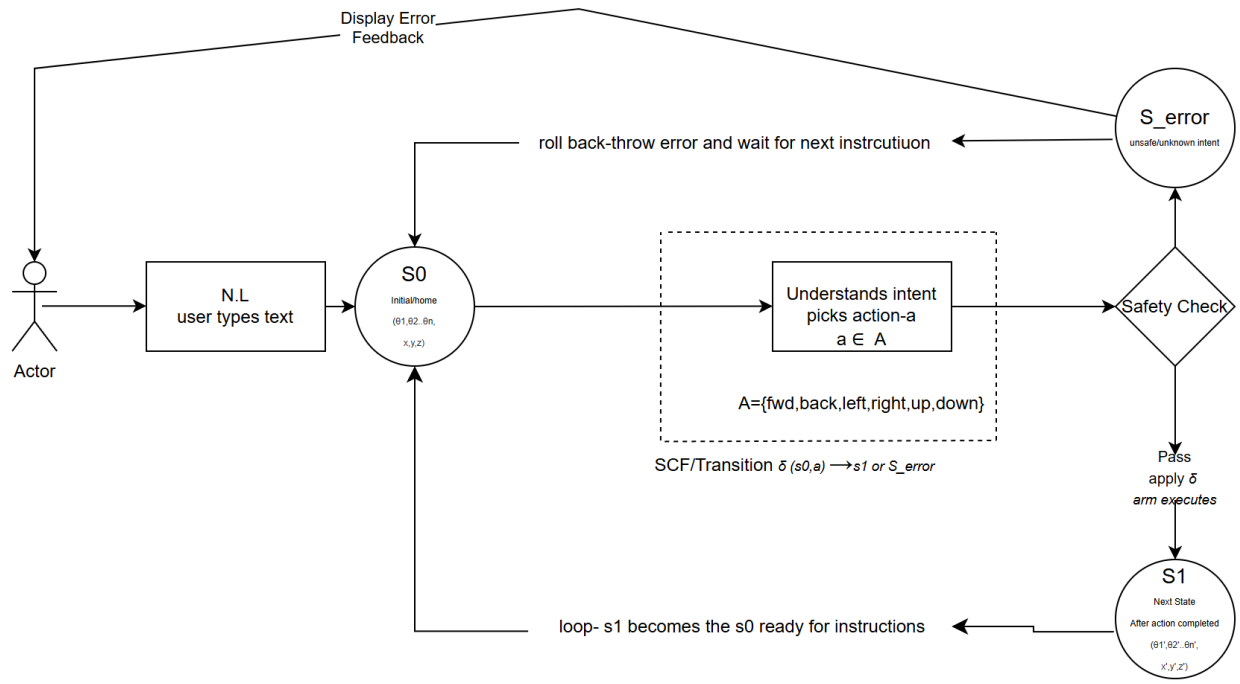
AI & Robotics Lab
Internal Research — April 2026

RoArm V0

Natural Language Control of a Robotic Arm — Architecture & Scalability

Hardware RoArm-M2-S (Waveshare, ESP32, 4-DOF) **Status** V0 complete — all 8 criteria verified on hardware

The Approach: Architecture First



State transition diagram

The core insight behind this project is that the architecture matters more than the intelligence layer, especially early on. A lot of NL-to-robot research jumps straight to large language models and end-to-end neural control. That produces impressive demos, but the systems are brittle and hard to reason about when something goes wrong.

V0 takes the opposite stance. Start with the simplest possible NL understanding (keyword matching), build the state-transition architecture around it correctly, verify every safety contract on real hardware, and only then swap in more capable components. The architecture becomes the stable scaffold that increasingly powerful modules plug into.

The State-Transition Model

Every action the arm takes is modelled as a formal state transition:

$$\delta(S_0, a) \rightarrow S_1 \quad \text{or} \quad S_{error}$$

$\mathbf{S_0}$ is the current state of the arm (its position in space). **a** is the action derived from what the user said. **δ** is the transition function — the pipeline that decides whether the action is safe and, if so, executes it. The arm either moves to the new state $\mathbf{S_1}$ or, if the action is unknown or would place the arm outside its safe envelope, transitions to **Serror** and recovers back to $\mathbf{S_0}$ automatically. There is no stuck state.

What makes this design useful is that the model doesn't care how the action was derived — regex today, a language model tomorrow, a vision system the day after. The transition function, the

safety checks, the state representation, and the hardware interface are all independent layers. Each one can be upgraded without touching the others.

How V0 Works

V0 targets a single, concrete capability: move the RoArm-M2-S in six directions (forward, back, left, right, up, down) using plain English instructions. That sounds modest, but the execution pipeline is the same one that will carry much more complex behaviour in later versions.

The Five-Step Pipeline

Every command, regardless of what the user types, passes through exactly five steps:

- **1 Classify** The user's text is classified into one of the six known actions — or returned as unknown if nothing matches.
- **2 Propose S'** A proposed new position is computed by applying the action's movement delta to the current position.
- **3 Safety check** Two safety gates run in sequence: is the action valid, and is the proposed position within the arm's safe workspace? If either fails, the arm does not move.
- **4 Execute** If safety passes, an HTTP command is sent to the arm's onboard ESP32 controller over WiFi. The arm moves.
- **5 Log & reset** The result is logged. On success the new position becomes the current state. On any failure the system resets cleanly to S_0.

One design choice worth highlighting: the proposed state is always computed on a copy of the current state, never directly. This means a safety failure cannot corrupt the system's record of where the arm actually is. The arm stays put, the state stays accurate, and the next command starts from a clean baseline.

V0 in Numbers

Hardware	RoArm-M2-S — Waveshare, ESP32-based, 4 degrees of freedom
Control interface	HTTP JSON over WiFi (arm runs its own web server)
NL understanding	Regex keyword matching — 6 directions, ~4–5 synonyms each
Avg command latency	~260ms round-trip (classify → send → arm confirms)
Accuracy on V0 set	87.5% (7/8 test phrases classified correctly)
Safety model	Hard workspace bounds: x 50–300mm, y ±200mm, z 50–350mm
State on failure	Auto-reset to S_0 — no manual recovery needed

Scalability

The honest answer is that V0 as built — regex patterns, fixed step sizes, six directions — does not scale. You cannot get to a general-purpose robot assistant by adding more regular expressions. At some point the vocabulary explodes, the patterns start conflicting, and the whole approach collapses.

But the architecture scales. That distinction is the whole point of starting here.

> *V0 is not the destination. It's the proof that the plumbing works. Every module is a slot — the interface is fixed, the implementation inside is replaceable.*

Why the Architecture Holds

The transition function δ doesn't care what's inside the AGENT box as long as it returns an action name. Right now that's a regex match. It could be a fine-tuned language model, a speech-to-intent classifier, or a multimodal system that interprets gesture and voice together. The rest of the pipeline doesn't change.

The same logic applies to every other layer. The safety checker currently enforces static workspace bounds. A future version can load a learned model of the arm's actual capabilities from its URDF — the Unified Robot Description Format, which encodes every link, joint, and movement limit of the physical robot — and run continuous collision checking. The interface stays the same.

High-level commands decompose naturally too. "Point north" isn't a V0 primitive, but it can be expressed as a sequence of left/right adjustments derived from the current orientation estimate. The V0 primitives become the instruction set that higher-level planners compile down to. This is the same principle compilers use — you don't rewrite the CPU every time you add a new programming language.

Version Roadmap

The table below shows how each layer of the architecture evolves across versions. The column structure reflects the four slots in the pipeline — each row is an independent upgrade path.

Version	NL Understanding	Safety & Control	Key Unlock
V0(now)	Regex keyword match	Static workspace bounds	Proves architecture end-to-end on hardware
V1	Small fine-tuned LLM (intent + magnitude)	URDF-aware joint limit checking	Natural phrasing, variable step size
V2	Full language model — context-aware	Learned workspace + obstacle detection	Multi-step commands, memory of prior actions

V3+	Multimodal (voice, gesture, vision)	Real-time collision model	Generalises to new robots via URDF swap

The URDF point in V3 is significant. Because the system's understanding of physical constraints is separate from the NL layer, swapping the robot means providing a new URDF — the arm's blueprint — not rewriting the control logic. The same architecture that runs on a 4-DOF desktop arm can, in principle, run on a 7-DOF collaborative robot with a different URDF and updated safety parameters.

What V0 Demonstrates

Eight criteria were defined before implementation started. All eight are verified on the physical hardware:

- All six direction commands classified correctly from natural language input
- Unknown or nonsense input routes to S_error — arm does not move
- Out-of-bounds commands blocked before reaching the arm
- WiFi failure handled gracefully — system recovers without manual reset
- S_error auto-resets to S_0 on every failure path
- Command latency under 500ms (measured average: ~260ms)
- Live accuracy and latency metrics available via web interface
- Browser UI shows real-time pipeline state per command

> *The most important thing V0 establishes is not what the arm can do — it's that every component can be replaced independently without breaking the system. That's the property that makes this architecture worth building on.*

Contact: iria@paraminnovation.org **Next milestone:** V1 — intent + magnitude via fine-tuned LLM